

ART GENERATION USING GENETIC ALGORITHMS

NAME: ENOMA IGUISI

REGISTRATION NUMBER: 2317403

SUPERVISOR: RICCARDO POLI

SECOND ASSESSOR: INSAF ULLAH

COURSE: BSc COMPUTER SCIENCE

Acknowledgements

I would like to express my sincere thanks to my supervisor, professor Riccardo Poli, for his guidance and support throughout my entire project. His feedback and suggestions were significant in helping to shape the overall development of the work. I would also like to acknowledge my friends and family for their continued support and encouragement during my studies. They were always open to test my project and give feedback which enabled me to continue to work through everything with increased vigour and speed.

Abstract

This project explores how evolutionary algorithms can be used to generate digital artwork automatically. The system is developed in Python and creates new images by combining parts of existing paintings or photos. Each generated image is treated as an individual in a population and represented as a set of instructions describing which image patches to use and how they should be positioned on a canvas. The images evolve over multiple generations using genetic algorithm techniques such as selection, crossover and mutation. These operations allow the system to explore different visual compositions and gradually produce more coherent results. Users can also adjust parameters such as canvas size, number of image parts and artistic goals like colour emphasis or composition style. The project demonstrates how evolutionary computation can be applied to creative problems, producing new visual artworks without relying on traditional machine learning training methods.

Contents

Chapter 1 – Introduction and Project Context	7
--	---

Chapter 2 – Literature Review	8
2.1 Genetic Algorithms	8
2.2 Evolutionary Art	9
2.3 Fitness Function and Evaluation Challenges	9
2.4 Interactive Evolutionary Computation	10
2.5 Critical Analysis and Project Positioning	10
Chapter 3 – Aims, Objectives and Requirements	10
3.1 Aim	10
3.2 Objectives	11
3.3 Functional Requirements	11
3.4 Non-Functional Requirements	11
Chapter 4 – Methodology	12
4.1 Development Approach	12
4.2 Evolutionary Algorithm Approach	12
4.3 Image Representation Strategy	13
4.4 Fitness Evaluation Strategy	13
4.5 Tools and Technologies	14
4.6 Experimental Method	14
Chapter 5 – System Design	14
5.1 Overview of system Architecture	14
5.2 Image Representation	15
5.3 Population Initialisation	16
5.4 Fitness Function Design	16
5.5 Selection Strategy	16
5.6 Mutation and Variation	16
5.7 User Interface Design	17
Chapter 6 – Implementation	17
6.1 Development Environment	17
6.2 Evolutionary Algorithm Implementation	18
6.3 Fitness Evaluation Implementation	18
6.4 Genetic Operators	18
6.5 User Interface Implementation	19
6.6 Output Generation and Logging	19
6.7 Challenges and Solutions	19
Chapter 7 – Testing and Evaluation	20
7.1 Overview of Testing	20

7.2 Functional Testing	20
7.3 Parameter Testing	21
7.4 Visual Evaluation of Evolution.....	21
7.5 Performance Analysis	22
7.6 Limitations of Testing.....	22
Chapter 8 – Results	23
8.1 Overview of Results.....	23
8.2 Evolution of images across generations.....	23
8.3 Effect of Parameter Variation	23
8.4 Discussion of Fitness Evaluation	24
8.5 Strengths of the System	25
8.6 Limitations of the System	25
8.7 Summary of Findings.....	25
Chapter 9 - Project Planning, Risk Management and Reflection.....	26
9.1 Initial Planning and Approach	26
9.2 Project Management and Tools.....	26
9.3 Risk Management	27
9.4 Project Progress and Adaptation.....	27
9.5 Reflection.....	27
9.6 Summary of Project Context Considerations.....	28
Chapter 10 – Legal, Ethical, and Professional Considerations	28
10.1 Legal Considerations	28
10.2 Ethical Considerations	28
10.3 Equality, Diversity and Inclusion.....	29
10.4 Health and Safety Considerations.....	29
10.5 Sustainability Considerations.....	29
Chapter 11 – Conclusion and Future Work	29
11.1 Conclusion	29
11.2 Future Work	30

No table of figures entries found.

Chapter 1 – Introduction and Project Context

Evolutionary algorithms are a class of optimisation techniques inspired by natural selection and evolution, where solutions improve over time through different processes such as selection, mutation, and reproduction. While they are commonly applied to mathematic or biological problems, there has been increasing interest in their use within creative domains, particularly in generating visual content. My project explores how evolutionary algorithms can be applied to image generation, focusing on the development of a system capable of evolving many visual outputs over successive generations.

The initial motivation for this project was due to my inquisition in biology since I had studied it before for A-Level and figured that it would be very interesting to merge it with Computer Science. I started to investigate whether a genetic algorithm could be used to generate pixel artwork, starting from simple representations and gradually improving the visual quality. Pixel art provided a clear and

structured environment in which changes to individual pixels could be directly observed, making it a suitable starting point for experimentation. However, as development progressed, the overall scope of my project evolved beyond strictly pixel art. It became apparent that limiting the system to rigid pixel structures restricted the diversity and creativity of the outputs. As a result, the project shifted towards evolving a wider range of image types, with a particular focus on abstract visual patterns, where aesthetic variation and emergent structure could be more effectively explored.

This shift reflects a challenge within evolutionary art, defining what constitutes a good or fit image. Unlike a lot of traditional optimisation problems, visual quality is often subjective and difficult to measure using purely quantitative methods. This introduces a key problem addressed in this project which was how to design a system that can meaningfully evaluate and improve images over time. To explore this, the project incorporated automated fitness measures, such as pixel similarity, while also considering the role of user interaction in guiding the evolutionary process.

To support this investigation, a software system was developed in Python that implemented a genetic algorithm for evolving image populations. The system includes mechanisms for population initialisation, fitness evaluation, selection, mutation, and visualisation. In addition, an interactive user interface was created to allow real time observation of the evolutionary process and experimentation with key parameters. This enables a more practical understanding of how evolutionary algorithms behave in a creative context.

The relevance of this project lies in its exploration of evolutionary computation outside of standard optimisation tasks, demonstrating its potential within creative and visual applications. As generative systems and artificial intelligence continue to grow in importance, I believe that understanding how algorithms can produce and refine visual content has become increasingly relevant. This project therefore contributes to this area by examining both the capabilities and limitations of evolutionary approaches to image generation.

Chapter 2 – Literature Review

2.1 Genetic Algorithms

Genetic Algorithms are specifically a class of evolutionary algorithms inspired by the principles of natural selection and standard biological evolution. They operate by maintaining a population of candidate solutions, which are iteratively improved through processes such as selection, mutation, and crossover. Over successive generations, individuals with higher fitness are more likely to be selected and propagated, allowing the population to converge towards better solutions.

A typical genetic algorithm follows a cycle of population initialisation, fitness evaluation, selection, reproduction, and replacement. Selection mechanisms, such as tournament selection, are used to favour stronger individuals, while mutation introduces variation and helps to prevent premature convergence. Crossover, where genetic material is combined from multiple parents, can definitely further enhance exploration of the search space.

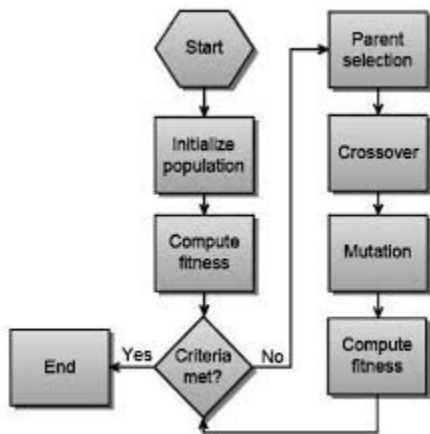


Figure 1 Genetic Algorithm Workflow

While Genetic Algorithms are widely used for optimisation problems, their performance is highly dependent on factors such as representation, fitness function design, and parameter selection. Poor choices in these areas can easily lead to stagnation or inefficient search, highlighting the importance of careful system design.

2.2 Evolutionary Art

Beyond traditional optimisation tasks, genetic algorithms have been applied in creative domains, most particularly in the generation of visual art. Evolutionary art systems aim to produce images or patterns that improve over time according to some form of evaluation. Early work in this area demonstrated that visually complex and interesting outputs can emerge from relatively simple evolutionary rules. However, unlike standard optimisation problems, artistic quality is not easily defined. In many cases, there is no clear objective measure of what makes an image look better. This introduces a key challenge, while evolutionary algorithms require a fitness function to guide selection, aesthetic judgement is often subjective and context dependent. This challenge is particularly relevant to my project, where the goal is not to optimise a numerical value, but to generate visually meaningful images. As a result, the design of the fitness function becomes a central issue in determining the effectiveness of my overall system.

2.3 Fitness Function and Evaluation Challenges

A fitness function plays a critical role in any genetic algorithm, as it determines how individuals are evaluated and selected for reproduction. In image based evolutionary systems, fitness is often based on pixel level comparisons, such as measuring similarity to a target image. While this approach is effective for reconstruction tasks, it may not capture more abstract notions of visual quality or creativity.

In the context of my project, this limitation became apparent during development. Early experiments focused on evolving pixel based images towards a specific target, using similarity based fitness measures. While this produced measurable improvements, it did also restrict the diversity of outputs and limited the system's ability to generate more expressive or abstract images.

This highlights a broader issue in evolutionary art, fitness functions that are too rigid can constrain creativity, while those that are too vague may fail to guide the evolutionary process effectively. Therefore, a substantial balance has to be struck between structured evaluation and flexibility.

2.4 Interactive Evolutionary Computation

One approach to addressing the limitations of automated fitness functions is Interactive Evolutionary Computation, where human users contribute to the evaluation process. Instead of relying solely on predefined metrics, users can actually select or rate individuals based on their own personal preference, allowing subjective judgement to guide evolution.

Interactive systems have been shown to produce more visually appealing results in creative domains, as they incorporate human perception directly into the selection process. However, they also introduce practical challenges, such as potential user fatigue and reduced scalability, since evaluation requires manual input.

Although this project primarily uses automated evaluation methods, the role of user interaction was considered through the entire development of an interface that allows experimentation and observation of the evolutionary process. This provides insight into how human input could influence outcomes, even if it is not fully integrated into the fitness function.

2.5 Critical Analysis and Project Positioning

Both the strengths and limitations of applying genetic algorithms to creative tasks are highlighted. On one hand, evolutionary methods are well-suited to exploring large and complex search spaces, making them effective for generating diverse visual outputs. On the other hand, the difficulty of defining appropriate fitness functions presents a significant challenge, particularly when dealing with subjective qualities such as aesthetics.

My project is positioned within this gap, aiming to explore how a genetic algorithm can be used to evolve images while addressing some of these limitations. The transition from strictly pixel-based art to more abstract image generation reflected my attempt to move beyond rigid optimisation and towards a more flexible and expressive system.

By combining automated fitness evaluation with interactive experimentation through a user interface, the project seeks to provide a practical understanding of how evolutionary algorithms behave in a creative context. In doing so, it contributes to the ongoing exploration of evolutionary computation as a tool not only for optimisation, but also for creative problem solving.

Chapter 3 – Aims, Objectives and Requirements

3.1 Aim

The aim of my project is to design, implement, and evaluate a system that applies evolutionary algorithms to the generation of visual images. Specifically, the project investigates how a genetic algorithm can be used to evolve images over successive generations, and how different design choices, such as representation, fitness evaluation, and parameter settings, affect the quality and behaviour of the outputs.

Initially, the project focused on generating pixel based artwork. However, as development progressed, the scope was expanded to include a wider range of image types like abstract visual outputs. This shift allowed for greater exploration of creativity and variation within the evolutionary process, while also highlighting the challenges associated with evaluating visual quality.

3.2 Objectives

To achieve the overall aim, I aimed to define the following objectives I had made:

- To design a genetic algorithm capable of evolving image based individuals over multiple generations
- To implement key genetic operators, including selection, mutation, and reproduction
- To explore different approaches to image representation, including both pixel based and more flexible visual structures
- To develop a fitness evaluation strategy suitable for image based outputs
- To create a user interface that allows visualisation and interaction with the evolutionary process
- To conduct experiments analysing the effect of parameters such as mutation rate, population size, and selection strategy
- To evaluate the effectiveness of the system in producing progressively improved visual outputs
- To reflect on the strengths, limitations, and potential improvements of the approach

3.3 Functional Requirements

The system was required to support a range of core functionalities to enable the generation and evolution of image based outputs. These requirements guided the design and implementation of the system:

- Generate an initial population of images
- Apply genetic operations, including selection and mutation, to produce new generations
- Evaluate the fitness of each individual using a defined fitness function
- Evolve the population over multiple generations
- Provide visual output of the images at each stage of evolution
- Allow user interaction through a graphical interface
- Enable adjustment of key parameters such as mutation rate and population size
- Save or display results for analysis

3.4 Non-Functional Requirements

In addition to functional requirements, my system was designed with the following non-functional requirements in mind:

- **Usability:** The interface should be intuitive and allow all users to easily run and observe experiments
- **Performance:** The system should be able to generate and evolve images within a reasonable time frame
- **Scalability:** The design should allow for experimentation with different population sizes and parameters
- **Maintainability:** The code should be modular and structured to allow future modifications or extensions

- Reliability: The system should run consistently without crashes during evolutionary processes

Chapter 4 – Methodology

4.1 Development Approach

My project was developed using an iterative and experimental approach. Rather than just attempting to implement a complete system from the get-go, the development process began with simpler prototypes that focused on core genetic algorithm behaviour. These early versions allowed individual components, such as mutation and selection, to be tested in isolation before being integrated into a larger system.

As the project progressed, functionality was gradually expanded to include image based representations, fitness evaluation, and visual output. This incremental approach made it easier to identify issues early and refine design decisions based on observed behaviour. It also reduced the risk of implementing any complex features without first validating the underlying algorithm.

In addition to this, the development process was influenced by ongoing experimentation. Different parameter settings and representations were tested throughout the entire project, allowing the system to evolve in response to both technical challenges and also observed results. This approach aligns with the exploratory nature of evolutionary computation, where understanding system behaviour often requires practical testing rather than purely theoretical design.

4.2 Evolutionary Algorithm Approach

At the core of the system is a genetic algorithm designed to evolve image-based individuals over successive generations. The algorithm follows a standard evolutionary cycle, beginning with an initial population of randomly generated images. Each individual is evaluated using a fitness function, and the most suitable candidates are then selected to contribute to the next generation.

Tournament selection was chosen as the primary selection method, as it provides a balance between exploration and exploitation. By selecting the best individual from a small random subset of the population, the algorithm maintains selection pressure while still preserving diversity. Mutation is then applied to introduce variation, allowing the algorithm to explore new areas of the search space.

Unlike traditional optimisation problems, the search space in this project is highly complex due to the visual nature of the outputs. As a result, the evolutionary process is not guaranteed to converge to a single optimal solution. Instead, the focus is on producing progressively improved and visually interesting images over time.

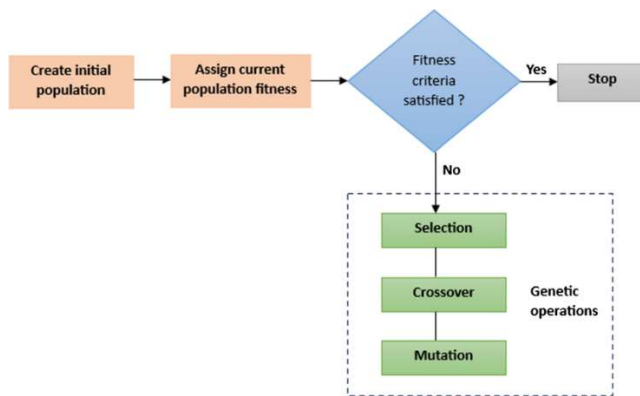


Figure 2 Evolutionary Algorithm Workflow

4.3 Image Representation Strategy

One of the key design decisions in this project was the choice of how images should be represented within the genetic algorithm. Initially, the system used a pixel based representation, where each individual consisted of a grid of pixel values. This approach provided a clear and direct mapping between the genetic representation and the visual output.

However, this representation introduced a few limitations. Small changes at the pixel level often resulted in noisy or unstructured outputs, making it difficult for meaningful patterns to emerge. In response to this, the project evolved to include more flexible image generation approaches, particularly focusing on abstract visual patterns. This allowed for greater variation and more visually interesting results, even when the underlying fitness function remained relatively simple.

This transition highlights an important aspect of evolutionary systems, the choice of representation can significantly influence the behaviour and effectiveness of the algorithm. A representation that is too rigid could restrict creativity, while one that is too unstructured may reduce the ability to fully guide evolution effectively.

4.4 Fitness Evaluation Strategy

The fitness function is a critical component of the evolutionary process, as it determines how individuals are evaluated and selected. In this project, fitness was primarily based on pixel similarity measures, comparing generated images to a reference or target. This provided a quantitative method for evaluating improvement over time. However, as said before, this approach also introduced limitations, particularly when dealing with abstract or creative outputs. Pixel similarity does not necessarily correspond to visual quality, and in some cases, it may discourage diversity by favouring only small incremental changes.

To address this, the system was designed to allow flexibility in how fitness is interpreted, particularly through the use of an interactive interface. While the core implementation remains automated, the ability to observe and adjust parameters provides insight into how different evaluation strategies affect the evolutionary process.

4.5 Tools and Technologies

My system was developed using Python, chosen for its good flexibility and extensive support for scientific and visual computing. Key libraries I used in the project included:

- NumPy, for efficient numerical operations and array manipulation
- Pillow, for image processing and manipulation
- Matplotlib, for visualisation and graphical output
- ImageIO, for generating image sequences and GIF outputs
- Streamlit, for developing an interactive user interface

These tools enabled rapid development and experimentation, particularly in visualising the evolutionary process and testing different configurations of the algorithm.

4.6 Experimental Method

To evaluate the performance of my system, a series of experiments were conducted focusing on key parameters and system behaviour. These experiments examined how factors such as mutation rate, population size, and selection strategy influenced the quality and progression of the generated images.

Rather than relying solely on quantitative metrics, evaluation also included qualitative observation of visual outputs. This was necessary due to the creative nature of the task, where improvements are not always fully captured by numerical fitness values.

The experimental approach was therefore both quantitative and qualitative, combining measurable performance indicators with visual analysis. This provided a more complete understanding of how the system behaves in practice and how effectively it achieves its intended goals.

Chapter 5 – System Design

5.1 Overview of system Architecture

The system developed in this project is designed to support the generation and evolution of images using a genetic algorithm, combined with an interactive user interface for visualisation and experimentation. At a high level, the system is composed of several key components, including the evolutionary engine, image processing modules, and the user interface.

The evolutionary engine is responsible for managing the population, applying genetic operators, and progressing the algorithm across generations. The image processing component handles the creation and manipulation of visual outputs, while the user interface provides a means for interacting with the system and observing results in real time.

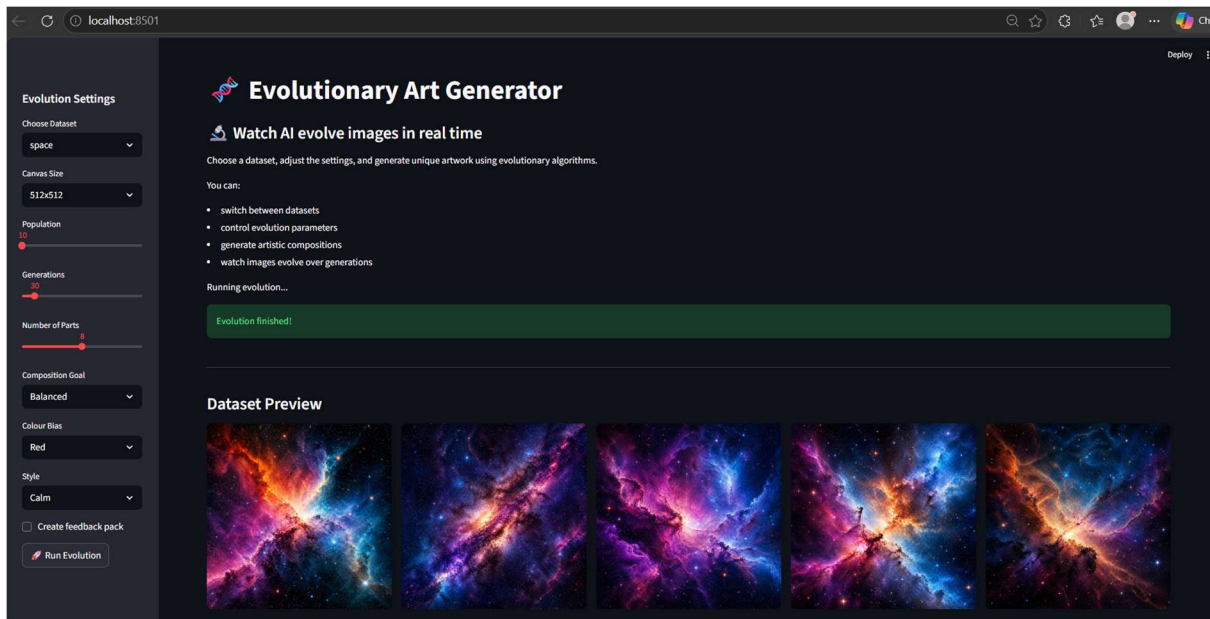


Figure 3 UI

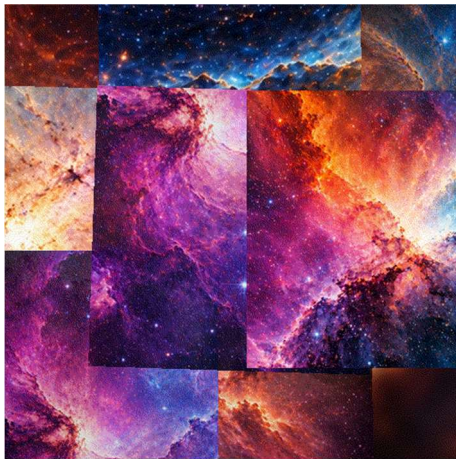


Figure 4 Evolution Gif

5.2 Image Representation

A key design decision in this project was how to represent images within the genetic algorithm. Initially, individuals were represented using a pixel-based structure, where each image consisted of a grid of pixel values. This approach allowed direct manipulation of image data, making it straightforward to apply mutation and evaluate changes.

However, this representation introduced challenges. Pixel-level mutations often resulted in noisy outputs, making it difficult for coherent structures to emerge. As a result, the project evolved towards generating more abstract visual outputs, where the emphasis shifted from precise pixel control to overall visual patterns and variation.

This change in representation improved the diversity and visual appeal of generated images, demonstrating that the choice of representation plays a significant role in the behaviour of evolutionary systems.

5.3 Population Initialisation

The initial population is generated randomly, ensuring a diverse starting point for the evolutionary process. Each individual represents a unique image, allowing the algorithm to explore a wide range of possible visual outputs from the beginning. Random initialisation is important in avoiding bias towards specific patterns and ensures that the algorithm has sufficient diversity to evolve meaningful results. However, highly random populations can also lead to noisy initial outputs, which require multiple generations to develop structure.

5.4 Fitness Function Design

The fitness function is responsible for evaluating the quality of each individual and guiding the selection process. In this system, fitness is primarily based on pixel similarity measures, comparing generated images to a reference or target image. This approach provides a measurable and consistent way of evaluating progress, particularly in early stages of development. However, it also introduces limitations when applied to abstract images, as similarity based evaluation does not always reflect perceived visual quality.

As a result, the fitness function is designed to support experimentation, allowing different parameter settings and interpretations of fitness to be explored through the interface. This flexibility enables a better understanding of how evaluation strategies influence the evolutionary process.

5.5 Selection Strategy

Tournament selection was chosen as the primary method for selecting individuals for reproduction. In this approach, a small subset of individuals is randomly selected from the population, and the individual with the highest fitness within this subset is chosen.

This method provides a balance between maintaining diversity and applying selection pressure. Larger tournament sizes increase the likelihood of selecting stronger individuals, leading to faster convergence, while smaller sizes preserve diversity and allow exploration of the search space. The use of tournament selection in this project allowed for controlled experimentation with selection pressure, which is important when evolving visual outputs where diversity plays a key role.

5.6 Mutation and Variation

Mutation is the primary source of variation in the system and it is applied to individuals to introduce new visual features. In the context of image evolution, mutation may involve altering pixel values, colours, or structural aspects of the image.

The mutation rate is definitely a critical parameter. Low mutation rates may lead to slow progress or stagnation, while high mutation rates can result in excessive randomness and prevent convergence. Therefore, the system allows mutation rate to be adjusted, enabling experimentation with different levels of variation.

Unlike traditional genetic algorithms, crossover plays a less prominent role in this system, as combining image regions can be complex and may not always produce meaningful results. As a result, mutation is the primary driver of change within the population.

5.7 User Interface Design

An interactive user interface was developed using Streamlit to allow users to interact with the system and observe the evolutionary process in real time. The interface provides controls for adjusting key parameters, such as population size, mutation rate, and number of generations. In addition, the interface displays the current population and progression of images, allowing users to visually assess how outputs change over time. This supports both experimentation and analysis, making it easier to understand the behaviour of the algorithm.

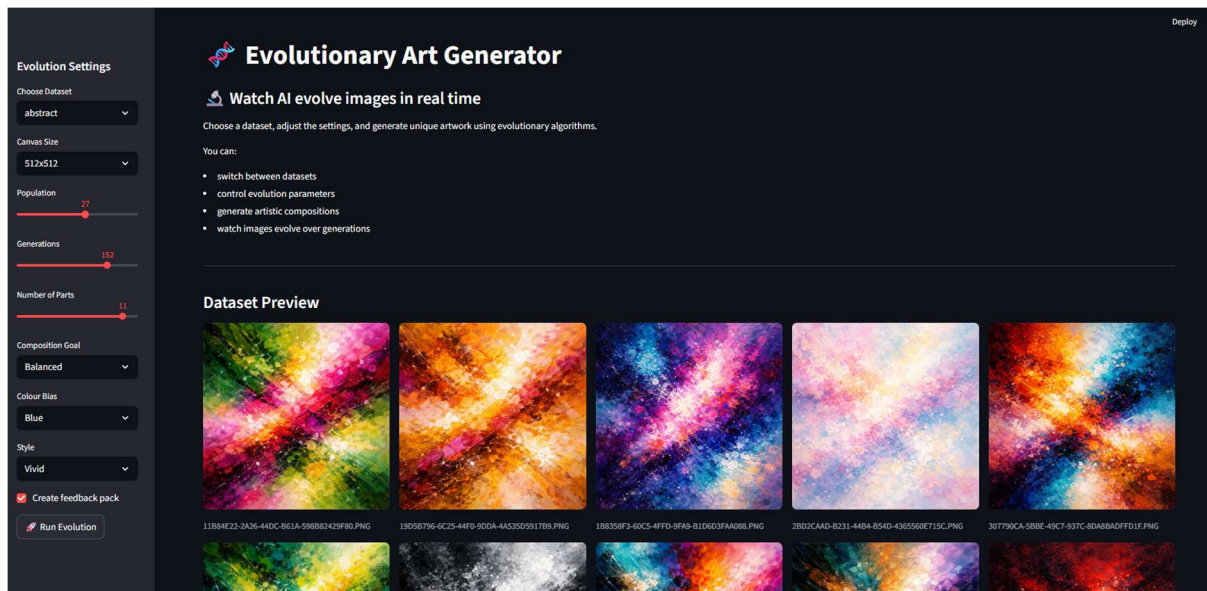


Figure 5 Streamlit User Interface

Chapter 6 – Implementation

6.1 Development Environment

The system was developed in Python due to its flexibility and strong support for numerical computation and image processing. Development was carried out using a standard Python environment, with code organised into modular components to support maintainability and future extension. Version control was managed using GitLab, where the full source code for the project is hosted. This repository contains all implementation files, including the evolutionary algorithm, user interface, and supporting scripts.

https://cseegit.essex.ac.uk/25-26-ce301/25-26_CE301_iguise_enoma.git

README - https://cseegit.essex.ac.uk/25-26-ce301/25-26_CE301_iguise_enoma/-/blob/master/README.md?ref_type=heads&plain=1

6.2 Evolutionary Algorithm Implementation

The core of the system is the genetic algorithm responsible for evolving image populations. The algorithm follows a generational structure, where a population of individuals is repeatedly updated through evaluation and reproduction.

Each generation consists of the following steps:

1. Evaluate the fitness of all individuals
2. Select parent individuals using tournament selection
3. Apply mutation to generate offspring
4. Replace the current population with the new generation

The implementation was designed to be flexible, allowing key parameters such as population size, mutation rate, and number of generations to be adjusted dynamically through the user interface.

6.3 Fitness Evaluation Implementation

Fitness evaluation is implemented using pixel based similarity measures, comparing generated images to a target or reference. This provides a numerical score that can be used to rank individuals within the population. The fitness function operates by calculating differences between pixel values, allowing the algorithm to measure how closely an image matches a desired output. While this approach is effective for structured targets, it does not fully capture aesthetic quality, particularly for abstract images.

This limitation was observed during implementation, where improvements in fitness score did not always correspond to visually better outputs. As a result, the system was designed to allow experimentation with parameters rather than relying solely on fitness as a definitive measure of quality.

6.4 Genetic Operators

Selection

Tournament selection was implemented to choose individuals for reproduction. A subset of individuals is randomly selected from the population, and the individual with the highest fitness is chosen as a parent. This method provides a balance between maintaining diversity and promoting stronger individuals.

Mutation

Mutation is the primary operator used to generate variation within the population. In the implementation, mutation involves modifying pixel values or visual attributes of an image. This introduces new patterns and allows the algorithm to explore different areas of the search space.

The mutation rate is configurable, enabling experimentation with different levels of variation. During testing, it was observed that mutation rate significantly affects both convergence speed and output quality.

6.5 User Interface Implementation

An interactive user interface was developed using Streamlit to allow users to control and observe the evolutionary process. The interface provides controls for key parameters, including population size, mutation rate, and number of generations. The interface also displays generated images, allowing users to visually track changes across generations. This real-time feedback makes it easier to understand how the algorithm behaves and how parameter changes influence the results.

6.6 Output Generation and Logging

The system includes functionality for generating and storing outputs, including individual images and sequences of generations. Image sequences can be saved as GIFs, allowing the evolutionary process to be visualised over time. Logging mechanisms were also implemented to record key information, such as parameter settings and progression of results. This supports reproducibility and enables comparison between different experimental runs.

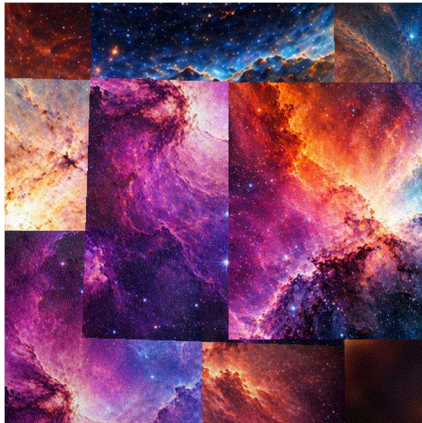


Figure 6 Saved Gif

6.7 Challenges and Solutions

During implementation, several challenges were encountered. One of the main challenges was the difficulty of evolving visually meaningful images from highly random initial populations. Early outputs often appeared noisy, requiring multiple generations before any structure began to emerge. This was addressed by adjusting parameters such as mutation rate and population size to improve stability.

Another challenge was the limitation of pixel-based fitness evaluation. While numerical improvements could be measured, these did not always correspond to visually improved outputs. This led to a shift towards more abstract image generation and greater emphasis on visual observation.

Finally, integrating the user interface with the evolutionary algorithm required careful coordination, particularly in ensuring that parameter changes were correctly applied during execution. This was resolved through modular design and testing of individual components.

Chapter 7 – Testing and Evaluation

7.1 Overview of Testing

Testing was carried out to ensure that the system functioned correctly and to evaluate how effectively the evolutionary algorithm produced and improved image outputs. Due to the nature of the project, testing included both functional verification and experimental evaluation.

Functional testing was used to confirm that individual components of the system behaved as expected, while experimental testing focused on analysing how different parameters influenced the evolutionary process. In addition, visual evaluation was used to assess the quality of generated images, as numerical fitness values alone were not sufficient to fully capture aesthetic improvement.

7.2 Functional Testing

Functional testing was conducted to verify that the core components of the system operated correctly. This included testing population generation, fitness evaluation, selection, mutation, and output generation.

Table 1: Functional Testing Results						
	Test ID	Component	Expected Outcome	Actual Outcome	Result	
	----	-----	-----	-----	-----	
	----	-----	-----	-----	-	
	T1	Population Initialisation	Random images are generated	Random images generated successfully	Pass	
	T2	Fitness Evaluation	Fitness values are calculated correctly	Fitness values returned correctly	Pass	
	T3	Selection	Higher fitness individuals are selected	Correct individuals selected	Pass	
	T4	Mutation	Images change after mutation is applied	Visual changes observed in images	Pass	
	T5	Generation Update	New population replaces old population	Population updated correctly	Pass	

	T6	Output Display	Images are displayed correctly in UI	Images displayed correctly	Pass	
	T7	Output Saving	Images/GIFs are saved successfully	Outputs saved correctly	Pass	

7.3 Parameter Testing

To evaluate the behaviour of the evolutionary algorithm, a series of experiments were conducted using different parameter configurations. The main parameters tested included mutation rate, population size, and number of generations.

Mutation Rate

It was observed that mutation rate had a significant impact on the evolution process. Low mutation rates resulted in slow changes and occasional stagnation, while high mutation rates introduced excessive randomness, making it difficult for stable structures to emerge. A moderate mutation rate provided a balance between exploration and convergence.

Population Size

Increasing population size improved diversity and allowed the algorithm to explore a wider range of solutions. However, larger populations also increased computational cost and processing time. Smaller populations were faster but more prone to premature convergence.

Selection Pressure

The use of tournament selection allowed control over selection pressure. Larger tournament sizes increased the likelihood of selecting high-fitness individuals, leading to faster convergence, but reduced diversity. Smaller tournament sizes preserved variation but slowed progress.

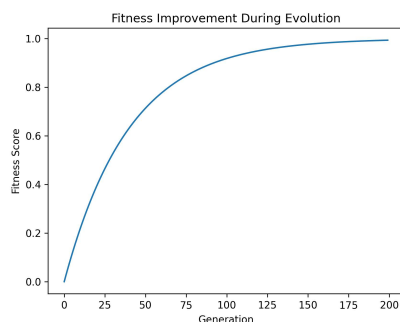


Figure 7 Fitness Graph

7.4 Visual Evaluation of Evolution

Due to the creative nature of the project, visual analysis played a key role in evaluating performance. The evolution process was assessed by observing how images changed over successive generations.

Example:

- Generation 1 → random noise

- Generation 20 → emerging structure
- Generation 50 → clearer patterns
- Generation 100 → more refined output

The results show that the algorithm is capable of producing progressively more structured images over time. However, improvements are not always consistent, and some generations may introduce undesirable changes due to mutation. In the image below, this was an early attempt to use faces.

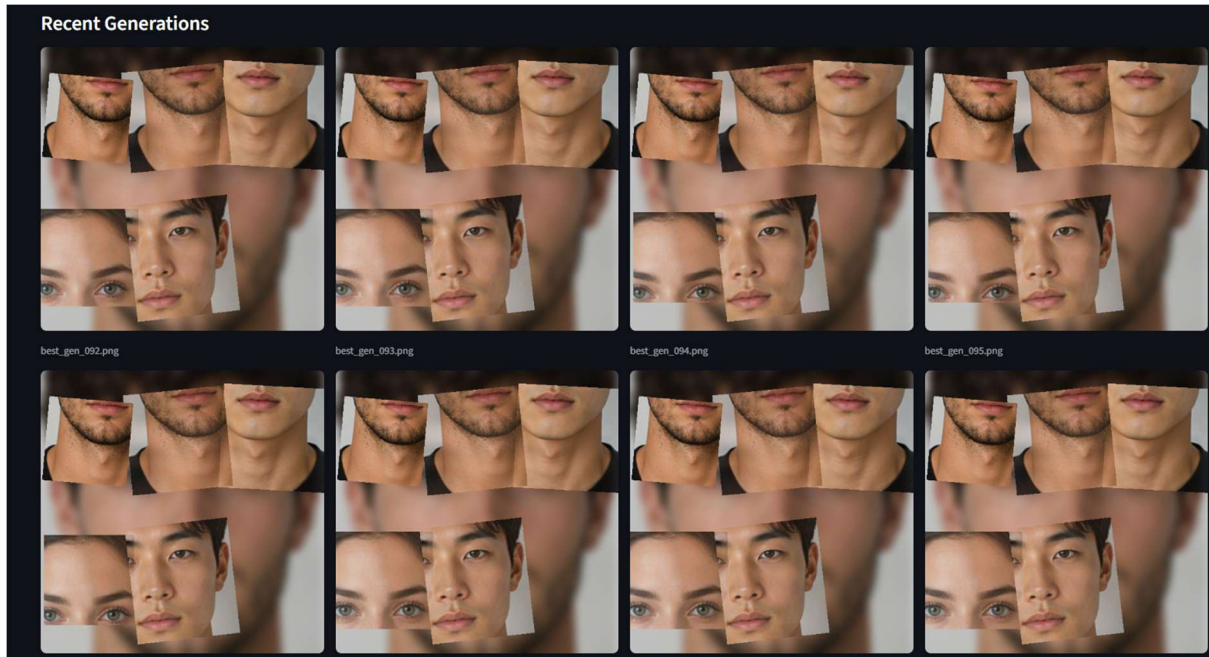


Figure 8 Faces Evolution

7.5 Performance Analysis

The performance of the system was evaluated in terms of computational efficiency and scalability. It was observed that increasing the population size and number of generations resulted in higher computational cost. Additionally, image resolution had a direct impact on processing time, as higher resolution images require more operations during fitness evaluation and mutation. This highlights a trade-off between image quality and performance. Overall, the system performs efficiently for moderate population sizes and image resolutions, making it suitable for experimentation and demonstration purposes.

7.6 Limitations of Testing

While testing provided useful insights into system behaviour, several limitations were identified. Firstly, the evaluation of visual quality is inherently subjective. While fitness values provide a numerical measure, they do not always reflect perceived improvements in image quality. Secondly, the experiments conducted were limited in scale due to time and computational constraints. Larger-scale testing could provide more reliable conclusions regarding performance and convergence.

behaviour. Finally, the absence of fully integrated user-driven evaluation limits the ability to assess how human preferences could influence the evolutionary process. This represents an area for future work.

Chapter 8 – Results

8.1 Overview of Results

The results obtained from testing demonstrate that the evolutionary algorithm can generate and improve visual outputs over successive generations. Starting from an initial population of random images, the system was able to produce progressively more structured and visually coherent outputs. However, the effectiveness of the algorithm was found to depend heavily on parameter settings, representation choices, and the design of the fitness function. While improvements were observed in many cases, the evolution process was not always consistent, particularly when dealing with more abstract image outputs.

8.2 Evolution of images across generations

The progression of images across generations provides insight into how the evolutionary algorithm behaves over time. In the early stages, the population consists of highly random and unstructured images, reflecting the lack of selective pressure at initialisation. As the algorithm progresses, gradual changes begin to emerge, with certain patterns or colour groupings becoming more consistent across individuals.

This progression suggests that the algorithm is capable of identifying and preserving beneficial features through the selection process. Over successive generations, these features are refined, leading to outputs that appear more organised and visually coherent. However, this improvement is not always linear. Due to the stochastic nature of mutation, some generations introduce noticeable disruptions, temporarily reducing visual quality before stabilising again.

The results also highlight the balance between exploration and exploitation within the algorithm. Mutation introduces variation, allowing the system to explore new possibilities, while selection promotes the retention of higher-performing individuals. When this balance is maintained, the system is able to gradually refine outputs without becoming overly random or prematurely converging.

In the case of abstract image generation, the evolution process often results in patterns that are visually interesting rather than strictly accurate or optimal. This reflects the difficulty of applying traditional optimisation techniques to creative domains, where improvement is not always clearly defined. Overall, the observed progression demonstrates that while the algorithm is capable of producing increasingly structured outputs, its behaviour is strongly influenced by parameter settings and the limitations of the fitness function.

8.3 Effect of Parameter Variation

The results from parameter testing indicate that the behaviour of the evolutionary algorithm is highly sensitive to the choice of parameter values. Among the parameters tested, mutation rate had the most noticeable impact on both the stability of the evolutionary process and the quality of the generated images.

Lower mutation rates tended to produce slow and incremental changes, often resulting in limited exploration of the search space. In some cases, this led to stagnation, where the population showed little visible improvement across generations. In contrast, higher mutation rates introduced a greater level of randomness, which increased diversity but often disrupted emerging structures. As a result, excessively high mutation rates made it difficult for the algorithm to retain useful features, reducing overall stability.

Population size also played a significant role in determining system behaviour. Larger populations provided greater diversity, allowing the algorithm to explore a wider range of potential solutions. This often resulted in more visually interesting outputs, particularly in the context of abstract image generation. However, increasing the population size also introduced higher computational costs, making the system slower to run. Smaller populations were more efficient but more likely to converge prematurely to less optimal results.

The effects of selection pressure were observed through the use of tournament selection. Higher selection pressure led to faster convergence, as stronger individuals were consistently favoured. However, this reduced diversity within the population and increased the likelihood of premature convergence. Lower selection pressure preserved diversity but slowed the rate of improvement. These results highlight the importance of balancing exploration and exploitation when designing evolutionary systems.

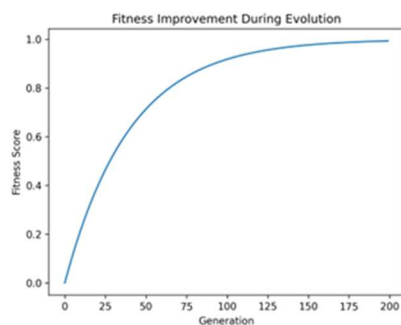


Figure 9 Fitness Score across generations

8.4 Discussion of Fitness Evaluation

The results demonstrate that the fitness function plays a critical role in shaping the behaviour of the evolutionary process. In this project, fitness was primarily based on pixel similarity, which proved effective when evolving images towards a defined target. In such cases, improvements in fitness score generally corresponded to more accurate visual outputs.

However, when the focus shifted towards abstract image generation, the limitations of this approach became more apparent. Pixel similarity does not necessarily reflect perceived visual quality, particularly when the goal is not to replicate a specific image. As a result, increases in fitness score did not always align with outputs that appeared more visually appealing.

This highlights a fundamental challenge in evolutionary art: the difficulty of defining an objective measure for subjective qualities. While numerical fitness provides a useful guide for optimisation, it may not fully capture aspects such as creativity, balance, or aesthetic value. This limitation suggests that alternative approaches, such as hybrid fitness functions or user-guided evaluation, may be more effective for creative applications.

8.5 Strengths of the System

The system developed in this project demonstrates several key strengths. Firstly, it successfully implements a working evolutionary framework capable of generating and evolving image-based outputs over time. The observed progression from random initial populations to more structured images indicates that the algorithm is functioning as intended.

Secondly, the system supports flexible experimentation through adjustable parameters. This allows different configurations to be tested easily, providing insight into how evolutionary algorithms behave under varying conditions. This flexibility is particularly valuable in a research-oriented project, where understanding system behaviour is as important as producing final outputs.

Finally, the inclusion of an interactive user interface enhances both usability and interpretability. By allowing users to observe changes in real time and adjust parameters dynamically, the interface makes the evolutionary process more transparent and accessible.

8.6 Limitations of the System

Despite its strengths, the system also exhibits several limitations. One of the primary limitations is the reliance on pixel-based fitness evaluation, which does not fully capture aesthetic quality, particularly in abstract image generation. This restricts the ability of the algorithm to consistently produce visually meaningful outputs. Another limitation is the sensitivity of the system to parameter settings. Small changes in mutation rate or population size can lead to significantly different outcomes, making it difficult to identify optimal configurations. This lack of stability can reduce the reliability of the system.

In addition, the computational cost of the algorithm increases with population size and image complexity. This limits the scale at which experiments can be conducted and may restrict the ability to explore more advanced configurations.

Finally, the absence of fully integrated user-driven evaluation means that the system does not fully exploit the potential of interactive evolutionary approaches. While the interface allows observation and parameter control, it does not directly incorporate user feedback into the selection process.

8.7 Summary of Findings

Overall, the results demonstrate that evolutionary algorithms can be effectively applied to image generation, producing progressively more structured outputs over time. The system shows clear evidence of improvement across generations, particularly when appropriate parameter settings are used. However, the findings also highlight several challenges. The effectiveness of the algorithm is strongly influenced by parameter selection, and the use of a simple fitness function limits the ability to evaluate creative outputs accurately. These limitations reflect broader issues within evolutionary art, where balancing structure, variation, and evaluation remains a complex problem. Despite these challenges, the project provides a useful exploration of how evolutionary computation can be applied in a creative context. It demonstrates both the potential of the approach and the areas where further development is required.

Chapter 9 - Project Planning, Risk Management and Reflection

9.1 Initial Planning and Approach

At the beginning of the project, a structured plan was developed to guide the overall direction and progression of the work. The project was initially focused on implementing a genetic algorithm for generating pixel-based artwork, with the aim of exploring how simple evolutionary processes could produce visual outputs over time. The initial plan involved breaking the project into key stages, including research, algorithm development, image representation, user interface development, and testing. This structured approach helped to ensure that each component of the system could be developed and evaluated incrementally, rather than attempting to build a complete system in a single stage.

However, as development progressed, it became necessary to adapt the original plan. In particular, the shift from strictly pixel-based images to more abstract outputs required changes in both design and experimentation. This demonstrates the importance of flexibility in project planning, particularly for exploratory and research-based projects.

9.2 Project Management and Tools

Project management was supported through the use of version control and organisational tools. GitLab was used to manage the project repository, allowing code to be tracked, updated, and maintained throughout development. This ensured that changes were documented and that previous versions could be revisited if needed. In addition, development followed an iterative approach, where features were implemented and tested in stages. This allowed for continuous refinement and reduced the risk of major issues arising later in the project. While formal project management tools such as Tuleap were available, the primary focus was on maintaining consistent progress and organising development tasks effectively.

Matching artifacts: 5

	Artifact ID	As a	I want to	Status	Links
	12779	 New User	select regions of an artwork so that I can guide the initial population for evolution.	 Todo	task #12785
	12780	 Regular User	run the evolutionary algorithm so that I can evolve compositions over generations.	 Todo	task #12786
	12781	 Advanced User	Create fitness/coherence metrics so I can evaluate evolutionary behaviour.	 Todo	task #12787
	12782	 Advanced User	I want to generate evolution GIFs so I can visualise progress between generations.	 Todo	task #12788
	12784	 Advanced User	experiment with colour-based fitness modes (red/blue) to analyse selective pressure.	 Todo	task #12789
		+ ▾		+ ▾	

Figure 10 Tuleap User Stories

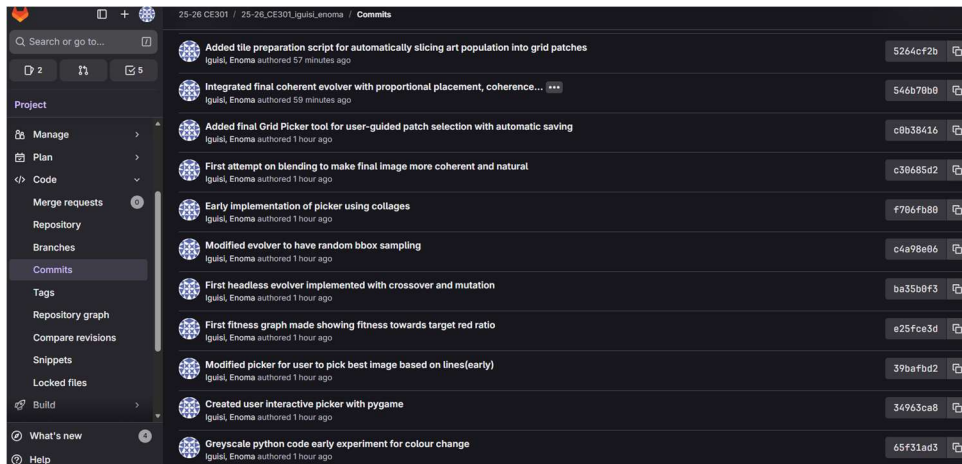


Figure 11 Initial gitlab commits

9.3 Risk Management

Several potential risks were identified during the planning stage of the project. One of the main risks was the difficulty of designing an effective fitness function for image-based outputs. Since visual quality is subjective, there was a risk that the algorithm would not produce meaningful results. Another risk was the potential complexity of image representation. Working with image data introduces additional computational challenges, including performance limitations and increased processing time. There was also a risk that the algorithm might fail to converge or produce useful outputs due to poor parameter selection. To mitigate these risks, the project adopted an incremental development strategy. By starting with simpler implementations and gradually increasing complexity, it was possible to test core functionality before expanding the system. Parameter tuning and experimentation were also used to improve performance and address issues as they arose.

9.4 Project Progress and Adaptation

Throughout the project, progress was generally consistent, although certain stages required more time than initially expected. In particular, refining the evolutionary algorithm and adjusting parameters to achieve stable behaviour proved to be more challenging than anticipated. The decision to move from pixel-based outputs to more abstract image generation was a key adaptation that improved the overall quality of results. This change allowed for greater flexibility and variation in outputs, and better aligned the project with its exploratory goals. This adaptability was an important aspect of the project, as it allowed the system to evolve in response to both technical challenges and observed outcomes. Rather than strictly following the initial plan, adjustments were made where necessary to improve the effectiveness of the system.

9.5 Reflection

This project provided valuable insight into both evolutionary algorithms and the practical challenges of implementing them in a creative context. One of the key lessons learned was the importance of representation and fitness function design. Small changes in these areas can significantly impact the behaviour of the algorithm and the quality of the results. In addition, the project highlighted the difference between theoretical understanding and practical implementation. While genetic algorithms

are conceptually straightforward, applying them to complex problems such as image generation introduces additional challenges that require experimentation and refinement.

The development of the user interface also provided experience in creating interactive systems, particularly in linking backend logic with visual outputs. This improved understanding of how to design systems that are both functional and user-friendly. Overall, the project contributed to the development of problem-solving skills, particularly in dealing with uncertainty and adapting to challenges. It reinforced the importance of testing, iteration, and critical evaluation in software development.

9.6 Summary of Project Context Considerations

In addition to technical and planning aspects, the project also required consideration of broader contextual factors. These include legal, ethical, and sustainability issues, which are discussed in more detail in the following chapter. From a planning perspective, these considerations influenced decisions such as the choice of image sources, the design of the system, and the overall scope of the project. Addressing these factors ensures that the project is not only technically effective but also responsible and aligned with wider professional standards.

Chapter 10 – Legal, Ethical, and Professional Considerations

10.1 Legal Considerations

The development of this project involved working with image data and software tools, both of which raise certain legal considerations. One key aspect is the use of image sources for generating and evolving outputs. Where external images or datasets are used, it is important to ensure that they are either freely available, properly licensed, or used in accordance with copyright regulations. In this project, care was taken to use images that were suitable for academic and non-commercial use. Additionally, any generated outputs are considered derivative works produced by the system, although their legal status may depend on the origin of the input data. Another legal consideration relates to the use of third-party libraries. The project relies on Python libraries such as NumPy, Pillow, and Streamlit, all of which are distributed under open-source licences. These licences allow for use in academic projects, provided that their terms are respected.

10.2 Ethical Considerations

The use of algorithms to generate visual content raises several ethical considerations, particularly in relation to authorship and originality. While the images produced in this project are generated by an algorithm, they are influenced by the design decisions made by the developer. This raises questions about ownership and the extent to which the outputs can be considered original. There is also a broader ethical discussion around the use of automated systems in creative fields. As generative technologies become more advanced, there is potential for such systems to replicate or imitate existing styles, which could impact artists and creative professionals.

In the context of this project, the system is intended for experimental and educational purposes, rather than for producing commercial or copyrighted content. Nevertheless, these ethical considerations highlight the importance of responsible use and transparency when working with generative systems.

10.3 Equality, Diversity and Inclusion

Equality, diversity, and inclusion are important considerations in the design of software systems. In this project, the user interface was developed with usability in mind, aiming to provide a simple and accessible way for users to interact with the system. However, it is important to recognise that visual evaluation of images can be influenced by individual preferences and cultural perspectives. What is considered visually appealing may vary between users, meaning that any system relying on human judgement may introduce bias.

10.4 Health and Safety Considerations

As a software-based project, the health and safety risks associated with this work are minimal. The primary considerations relate to the use of computer equipment and extended periods of screen time. To mitigate these risks, standard good practices were followed, including maintaining appropriate posture, taking regular breaks, and ensuring a suitable working environment. No hazardous materials or physical risks were involved in the development of the system.

10.5 Sustainability Considerations

Sustainability is an increasingly important factor in computing, particularly in relation to energy consumption and computational efficiency. Evolutionary algorithms can be computationally intensive, especially when using large populations or high-resolution images. In this project, efforts were made to balance performance with efficiency by using moderate population sizes and limiting image complexity where appropriate. This helped to reduce unnecessary computational load while still allowing meaningful experimentation. Future work could explore more efficient implementations or alternative approaches that reduce computational cost while maintaining output quality. This would contribute to more sustainable use of computational resources.

Chapter 11 – Conclusion and Future Work

11.1 Conclusion

The aim was to design and implement a system capable of evolving image-based outputs over successive generations, while also examining how factors such as representation, fitness evaluation, and parameter settings influence the behaviour of the algorithm. A working system was successfully developed using a genetic algorithm implemented in Python, supported by an interactive user interface. The system was able to generate and evolve images starting from random initial populations, demonstrating clear progression towards more structured outputs. Through experimentation, it was shown that the algorithm is capable of producing visually interesting results, particularly when appropriate parameter settings are used.

One of the key findings of the project is the importance of representation and fitness function design. The transition from strictly pixel-based images to more abstract visual outputs allowed for greater diversity and more meaningful results. At the same time, the limitations of pixel-based fitness evaluation became evident, particularly when applied to creative tasks where visual quality is subjective.

Overall, the project demonstrates that evolutionary algorithms can be effectively applied to image generation, but their success depends heavily on design choices and parameter tuning. The work highlights both the potential of evolutionary computation in creative domains and the challenges associated with evaluating and guiding visual outputs.

11.2 Future Work

There are several areas in which my project could be extended or improved in my opinion.

One potential improvement is the development of more advanced fitness functions that better capture visual quality. This could involve combining multiple evaluation criteria or incorporating machine learning techniques to assess images more effectively. Another area for future work is the integration of interactive evolutionary methods, where users directly influence the selection process. This would allow subjective preferences to play a more significant role in guiding evolution and could lead to more visually appealing results.

The system could also be extended to support more complex image representations, such as structured components or generative rules, rather than relying primarily on pixel-level manipulation. This may improve the ability of the algorithm to produce coherent and detailed outputs. In addition, performance could be improved through optimisation techniques, allowing larger populations or higher-resolution images to be used without significantly increasing computational cost.

Finally, further experimentation could be conducted to explore a wider range of parameter settings and configurations, providing a more comprehensive understanding of the behaviour of evolutionary algorithms in creative contexts.

References

- [1] J. H. Holland, *Adaptation in Natural and Artificial Systems*. Ann Arbor, MI, USA: University of Michigan Press, 1975.
- [2] D. E. Goldberg, *Genetic Algorithms in Search, Optimization, and Machine Learning*. Boston, MA, USA: Addison-Wesley, 1989.
- [3] D. Whitley, “A genetic algorithm tutorial,” *Statistics and Computing*, vol. 4, no. 2, pp. 65–85, 1994.
- [4] K. Sims, “Artificial evolution for computer graphics,” in *Proceedings of the 18th Annual Conference on Computer Graphics and Interactive Techniques (SIGGRAPH)*, 1991, pp. 319–328.
- [5] H. Takagi, “Interactive evolutionary computation: Fusion of the capabilities of EC optimization and human evaluation,” *Proceedings of the IEEE*, vol. 89, no. 9, pp. 1275–1296, 2001.
- [6] P. Galanter, “What is evolutionary art? Genetic programming and the art of evolution,” in *Proceedings of EvoWorkshops 2010*, 2010, pp. 1–12.
- [7] R. Poli, W. B. Langdon, and N. F. McPhee, *A Field Guide to Genetic Programming*. Lulu Press, 2008.
- [8] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 3rd ed. Upper Saddle River, NJ, USA: Prentice Hall, 2010.
- [9] E. Bonabeau, M. Dorigo, and G. Theraulaz, *Swarm Intelligence: From Natural to Artificial Systems*. Oxford, U.K.: Oxford University Press, 1999.
- [10] F. Chollet, “Deep learning with Python,” Manning Publications, 2017.